VigiLens: Real-Time Traffic Violation Vision Pipeline

Garv Jindal*, **Harsh Lakshakar, Jayagna Patel**
Birla Institute of Technology and Science, Pilani
*garvjindal2@gmail.com
{f20230358, f20231030}@pilani.bits-pilani.ac.in

Abstract

As urban centers grow, monitoring and enforcing traffic rules via camera feeds becomes essential. Real-world traffic feeds, however, suffer from severe degradation due to weather, low-light, occlusion, and varying camera hardware. This report presents VigiLens, an end-to-end, real-time traffic violation detection pipeline designed to overcome these challenges. The system is architected into four distinct phases: a smart conditional image preprocessing pipeline employing zero-shot light enhancement (Zero-DCE) and CLAHE; an object detection module powered by YOLO11; a mathematically robust, modular rule engine for spatial violation inference; and an evidence generation phase utilizing hybrid multi-engine OCR cascades for difficult license plates. Operating via a unified Streamlit interface, VigiLens bridges the gap between sophisticated computer vision algorithms and actionable enforcement evidence, effectively detecting complex violations such as overcrowding, stop-line crossing, and helmet non-compliance without the computational overhead of stacking massive neural networks.

1 Introduction

Automated traffic violation detection systems face significant bottlenecks primarily due to the poor quality of incoming video feeds and the complex spatial relationships of traffic participants. Traditional systems often rely either on brittle, single-purpose heuristic models or attempt to stack multiple heavy neural networks (e.g., a separate deep learning model for helmet detection, another for stop-lines, another for vehicle detection). This approach cripples frame rates and makes edge deployment impossible.

VigiLens is designed to resolve this by utilizing an intelligent "router" approach for image preprocessing and relying on high-speed spatial mathematics rather than secondary neural networks for behavioral inference. This report details the implementation choices, mathematical foundations, and architectural design of VigiLens, comprehensively breaking down its four-phase pipeline and user interface.

2 Phase 1: Smart Image Preprocessing Pipeline

Real-world CCTV and traffic cameras operate in imperfect conditions: nighttime, rain, fog, and glare. Processing raw frames directly leads to a drastic drop in detection accuracy. Conversely, applying heavy enhancement to every frame is computationally wasteful and can distort perfectly good daylight footage.

To solve this, Phase 1 employs a **Conditional Routing Mechanism**. Before any heavy computation occurs, the system evaluates three image quality metrics in under 5 milliseconds using optimized OpenCV algorithms:

1. **Brightness Score:** Evaluated via mean grayscale intensity.
2. **Blur Variance:** Measured using the variance of the Laplacian.
3. **Contrast Score:** Calculated via Root Mean Square (RMS) contrast.

Based on these heuristics, the frame is routed to the appropriate enhancement module:

2.1 Low-Light Enhancement: Zero-DCE

If the Brightness Score falls below the acceptable threshold, the frame is processed by **Zero-DCE** (Zero-Reference Deep Curve Estimation). We chose Zero-DCE over traditional Gamma Correction because it is a lightweight PyTorch neural network that estimates pixel-wise light enhancement

curves without requiring paired training data. It adapts dynamically to varying levels of darkness, preserving color fidelity which is crucial for identifying vehicle colors downstream.

2.2 Contrast and Blur Correction

For frames suffering from fog, glare, or motion blur (common for fast-moving vehicles):

- **CLAHE (Contrast Limited Adaptive Histogram Equalization):** Applied to mitigate washed-out contrast while preventing the noise amplification that plagues standard histogram equalization.
- **Unsharp Masking:** A high-pass filter via Gaussian Blurring is applied to sharpen edges, aiding the OCR phase in reading blurry license plates.

2.3 Aspect-Preserving Letterboxing

YOLO models require specific square input dimensions (e.g., 640×640). Naively resizing a 1920×1080 frame squashes the image. Because our Phase 3 rule engine relies heavily on *aspect ratios* to determine violations (like hazardous loads), distortion would break the system. We implement strict letterboxing (padding with black borders) to resize the image while maintaining the original aspect ratio, ensuring mathematical integrity downstream.

3 Phase 2: Vehicle & Road User Detection

With the image properly conditioned, the system must localize traffic participants.

3.1 Model Selection: YOLO11 Large

We evaluated several object detection models and explicitly selected **YOLO11 Large (yolo11l)** for state-of-the-art spatial awareness. The defining factor for this choice was YOLO11's *C2PSA* (Cross-Stage Partial Spatial Attention) mechanism. In dense traffic, vehicles and persons frequently occlude one another. C2PSA allows the model to maintain focus on partially hidden objects better than earlier iterations like YOLOv8.

3.2 Class Filtering and Dynamic Thresholding

To optimize processing, the detection tensor is aggressively filtered to retain only relevant COCO classes: `car`, `motorcycle`, `bus`, `truck`, `person`, and `bicycle`.

Furthermore, the system employs dynamic confidence thresholding. In standard conditions, the threshold is strictly set (e.g., 0.5) to prevent false positives. However, if Phase 1 detects heavy rain or fog, the confidence threshold is dynamically lowered to recover vehicles that the YOLO model is less certain about, effectively balancing recall and precision contextually.

4 Phase 3: Modular Violation Rules Engine

The core innovation of VigiLens is its Modular Violation Rules Engine. Running a separate neural network to detect if a helmet is worn, or if a car crossed a line, is computationally expensive. Instead, VigiLens infers complex behaviors using high-speed spatial mathematics directly derived from YOLO's bounding boxes.

The engine uses a strict plugin architecture, allowing new rules to be added without modifying the core pipeline.

4.1 Overcrowding / Triple Riding

Implementation: This plugin dynamically calculates the Intersection-over-Union (IoU) and spatial overlap between detected `person` bounding boxes and `motorcycle` bounding boxes. **Rationale:** Instead of training a custom "triple riding" model, we match riders to bikes. The algorithm calculates the center points of all pedestrians. If a person's center falls within an expanded upper-boundary of a motorcycle box, they are designated as a "rider". If the count of associated riders for a single motorcycle exceeds 2, an overcrowding violation is flagged.

4.2 Stop-Line & Wrong-Way Driving

Implementation: Operators use the Streamlit UI to draw virtual polygon zones over the camera feed (e.g., a red box over a crosswalk). The engine uses OpenCV's `cv2.pointPolygonTest` to evaluate vehicle coordinates against these zones. **Rationale:** We specifically check the *bottom-center coordinate* of the vehicle's bounding box. This represents the tires touching the road. If we checked the center of the box, the vehicle's hood could cross the stop-line without triggering a violation, which is legally inaccurate. If the tire coordinate enters the restricted polygon during a red light, it triggers a violation.

4.3 Hazardous Loads

Implementation: Evaluates the aspect ratio ($Width/Height$) of truck and motorcycle bounding boxes. **Rationale:** Vehicles carrying dangerous lateral loads (like metal pipes sticking out the sides) have an abnormally wide aspect ratio compared to standard vehicle dimensions. By comparing the calculated ratio against a pre-calibrated Area Profile standard deviation, the system instantly flags anomalous loads without needing a specialized dataset.

4.4 Helmet Non-Compliance

Implementation: To detect missing helmets without false-flagging pedestrians walking on the sidewalk, the algorithm performs a highly targeted spatial search. **Rationale:** The plugin defines an *expanded head area* (the top 20% of the motorcycle bounding box, expanded slightly outwards). If a YOLO detection for a 'person' or 'no-helmet' class has its center point located specifically within this zone, it is successfully mapped as a non-compliant rider. This mathematical restriction guarantees that standalone pedestrians without helmets are safely ignored.

4.5 Red-Light Violation

Implementation: Evaluates the exact same virtual polygon math as the stop-line rule, but gates it via a `traffic_light_status == "RED"` condition. **Rationale:** This isolates logic. A vehicle crossing a crosswalk on a green light is safe; crossing the exact same polygon coordinates on a red light instantly triggers the engine.

4.6 Illegal Parking

Implementation: The engine supports both a "Suspected" still-image mode (vehicle simply inside a no-parking polygon) and a "Confirmed" tracking mode. **Rationale:** When provided with tracker data (`stationary_seconds`), the engine requires the vehicle's dwell time to exceed a strictly defined minimum threshold before logging an official parked violation, preventing false flags on vehicles slowed in traffic.

4.7 Wrong-Side Driving

Implementation: Calculates the cosine similarity between the vehicle's motion vector (derived from a tracker like DeepSORT) and the lane's configured allowed direction. **Rationale:** Polygon checks fail for wrong-side driving on curved roads. Vector math accurately determines if a vehicle is traveling in stark opposition to the flow of traffic regardless of camera perspective.

4.8 Seatbelt Non-Compliance

Implementation: Similar to the helmet rule, this searches for `no_seatbelt` detections strictly within the bounding box area of a four-wheeler's cabin. **Rationale:** Prevents false positives by ensuring the system does not infer a violation merely from the absence of a seatbelt detection class; it must affirmatively identify an occupant without a belt.

4.9 Speeding

Implementation: Consumes external speed metadata (from radar or video calibrations) and compares it against *class-specific* limits. **Rationale:** Different vehicles have different legal thresholds. A motorcycle and a heavy truck driving at 50 km/h might trigger different logic, since the engine dynamically checks the limit dictionary per vehicle class.

5 Phase 4: License Plate OCR & Evidence

Once a vehicle is flagged by the rules engine, the system must generate legally actionable evidence. Running license plate detection over a full 4K frame is exceptionally noisy and computationally expensive.

5.1 Targeted Cropping

VigiLens implements a targeted cropping strategy. We extract only the region of interest defined by the violating vehicle's bounding box. The License Plate detection algorithm is then run *only* on this high-resolution crop, drastically improving accuracy and reducing inference time.

5.2 Hybrid OCR Cascading Strategy

Indian license plates are notoriously difficult to read due to non-standard fonts, dirt, damage, and sharp camera angles. A single OCR engine is insufficient. We designed a cascading hybrid OCR strategy to guarantee accuracy:

1. **Online Visio-LLMs (Primary):** We utilize Gemini Vision and Google Cloud Vision APIs. These multimodal models possess deep semantic understanding and can infer severely

degraded characters better than any traditional OCR.

2. **Offline Fallbacks (Secondary):** If the APIs are unreachable or timeout, the system falls back to a dual-engine local setup using **EasyOCR** and **PaddleOCR**. PaddleOCR provides excellent directional text detection, while EasyOCR excels at character extraction.

5.3 Regex Validation

To prevent false evidence, every OCR string is passed through a strict Regular Expression (Regex) validator tuned to Indian Regional Transport Office (RTO) formats (e.g., `^[A-Z]{2}[-\s]?\d{2}[-\s]?[A-Z]{1,2}[-\s]?\d{4}$`). If a read fails the regex, the cascade falls back to the next engine, ensuring only high-confidence plates are logged.

6 Unified Backend & Streamlit Interface

A robust backend orchestrates the four processing phases, managed entirely through a polished **Streamlit Frontend** tailored for traffic enforcement operators. The application acts as both a configuration hub and an evidence review dashboard.

6.1 Interactive Area Configurations

Traffic rules change depending on location. A speed of 60 km/h is legal on a highway but a violation in a school zone. The Streamlit app allows operators to define contextual "Area Profiles". These profiles inject priors into the Rules Engine on the fly. Furthermore, operators can interactively draw virtual polygons directly over the camera feed within the UI to define stop-lines, crosswalks, or no-parking zones.

6.2 Real-Time Analytics Dashboard

The interface features a live metrics dashboard powered by the underlying SQLite database. It provides statistical breakdowns of violation frequencies, peak hours, and recurring offense types, giving urban planners data-driven insights into traffic bottlenecks.

6.3 Human-in-the-Loop (HITL) Evidence Review

Fully automated systems are prone to edge-case failures. VigiLens mitigates this by maintaining a queue where all flagged violations are stored. The Streamlit app provides a dedicated "Human Review" tab. Here, an operator views the cropped evidence image, the bounding box overlays, and the extracted OCR text, and can choose to either *Approve* or *Reject* the violation before an official challan (ticket) is issued.

7 Limitations

While VigiLens offers a highly modular approach, several architectural and data-driven limitations remain:

- **Lack of Domain-Specific Training Data:** The system heavily relies on pre-trained COCO weights for YOLO11. Due to the lack of access to a large, annotated dataset of Indian traffic conditions during development, the base model occasionally struggles with unique local vehicle types (e.g., auto-rickshaws, loaded tractors).

- **License Plate Detection Heuristics:** Because we lacked sufficient data to fine-tune a YOLO model exclusively for detecting Indian license plates, the plate localizer relies heavily on computer vision heuristics (edge detection, contour filtering) within the vehicle crop. This fallback is susceptible to failure if the bumper is heavily textured, damaged, or obscured.

- **2D Spatial Ambiguity:** The rules engine calculates logic based on 2D bounding boxes. This causes issues with perspective occlusion. For example, a pedestrian standing on the sidewalk directly behind a parked motorcycle might have a bounding box that mathematically overlaps with the bike, falsely triggering a "rider" association.

8 Future Improvements

To evolve VigiLens towards a production-ready deployment, future iterations will focus on:

- **Dataset Aggregation & Fine-Tuning:** Collecting a robust dataset of Indian traffic and fine-tuning the YOLO11 model to natively detect license plates and region-specific vehicles (e.g., e-rickshaws, tractors).

- **Temporal Tracking Integration:** Implementing a multi-object tracking algorithm like **ByteTrack** or **DeepSORT**. This would allow the engine to evaluate behavior over time (e.g., distinguishing between a car that is parked

vs. one that is slowly moving through traffic) rather than relying purely on single-frame heuristics.

- **Edge Optimization:** Compiling the Zero-DCE and YOLO models using TensorRT to vastly accelerate inference speeds on edge devices such as NVIDIA Jetson modules.

9 Commercial Viability & Path to Productization

Beyond its immediate technical achievements, VigiLens is architected with clear commercial scalability in mind. It addresses the core financial and operational bottlenecks that typically prevent smart-city solutions from reaching production:

- **Hardware Agnosticism (Low CAPEX):** VigiLens is designed to consume standard RTSP video feeds from existing CCTV networks. Municipalities do not need to rip-and-replace their current infrastructure with expensive, proprietary smart cameras.

- **Edge Deployment Readiness:** By utilizing spatial heuristics instead of heavy, stacked neural networks for behavior tracking, the system demands significantly less VRAM. This allows it to be compiled via TensorRT and deployed on low-power edge devices (such as NVIDIA Jetson modules) directly at the intersection, saving massive cloud bandwidth and server costs.

- **Software-as-a-Service (SaaS) Modularity:** Because every traffic rule is an isolated Python plugin, the system can be effectively monetized using a tiered SaaS model. A city could purchase a "Base Package" (e.g., Speeding and Stop-lines) and later subscribe to "Add-on Modules" (e.g., Helmet Detection or Hazardous Loads) without requiring the core pipeline to be retrained or re-deployed.

- **Zero-Code Calibration:** The Streamlit interface allows non-technical end-users, such as traffic police officers, to dynamically draw bounding polygons and set contextual "Area Profiles" (adjusting speed limits and constraints). It is built as a usable product for the end-user, not just an engineering script.

10 Conclusion

VigiLens delivers a mathematically efficient, hardware-aware architecture for modern traffic enforcement. By moving away from brute-force neural network stacking and instead combining zero-shot preprocessing with intelligent spatial heuristics and a cascading OCR setup, it presents a highly viable, real-time solution. The modular four-phase design ensures that the pipeline can continuously evolve, offering urban planners and law enforcement an invaluable tool for improving road safety and compliance.